

数据库系统原理实验—实验七

存储过程和函数实验

学　　号: 23336179

姓　　名: 马福泉

专　　业: 计算机科学与技术

课 程 教 师: 赖韩江

起 始 日 期: 2025年11月6日

结 束 日 期: 2025年11月11日

实 验 地 点: B202

目录

1 实验概述 **2**

2 实验报告 **3**

2.1 无参数的存储过程或函数 3

2.1.1 实验目标 3

2.1.2 代码实现 3

2.1.3 运行结果 3

2.2 有参数的存储过程或函数 3

2.2.1 更新图书表价格 3

2.2.2 查找学生所读图书 4

2.2.3 图书价格调整 5

2.2.4 查找师承关系 6

2.3 游标实现图书总价计算 7

2.3.1 实验目标 7

2.3.2 代码实现 8

2.3.3 运行结果 8

2.4 实验总结 9

2.4.1 遇到的问题及解决方案 9

2.4.2 实验收获 10

1　实验概述

本实验旨在掌握数据库存储过程和函数的定义与使用。实验内容包括：

1. **无参数的存储过程或函数**
   定义一个存储过程或函数，实现打印“本人学号+姓名”（如：张三+22111001）。

2. **有参数的存储过程或函数**

   (a) 首先将数据库中的图书表 book_price 更新为指定价格：

book_id	book_name	book_price
b0001	SQL Server 2012宝典	98.00
b0002	职称英语专用教材	36.00
b0003	中国通史	57.00
b0004	丰子恺儿童文学选集	19.80
b0005	英语同义词辨析词典	63.50
b0006	数据库基础与应用	33.00
b0007	微积分初步	78.50
b0008	ASP.NET从入门到精通	69.80

   (b) 定义一个存储过程或函数：找出某个同学（输入名字，如“谢嫣然”）所读的所有图书名。

   (c) 定义一个存储过程或函数：如果图书的价格小于等于35元，增加图书的价格为 book_price ×(1+discount)；如果图书价格大于55元，图书的价格打折，更改为 book_price ×(1-discount)。注意：不要互相影响。例如 discount=0.9，30元的图书更新为 30×(1+0.9)=57，到这里就结束，不要继续做 57×(1-0.9)。

   (d) 导师关系表如下：

MentorName	StudentName
Alice	Bob
Alice	Carol
David	Alice
Mary	David
Susan	Mary
John	Susan

图 1: 导师关系表截图

   (e) 给定导师关系表，输入姓名，找出该学生的师承关系（导师、导师的导师、……），并返回所有相关导师的姓名。例如，输入 Mary，输出为 Susan, John。

3. **游标**
   定义一个存储过程或函数，用游标的方式计算所有图书的总价。

2　实验报告

2.1　无参数的存储过程或函数

2.1.1　实验目标

定义一个存储过程或函数，实现打印”本人学号+姓名”。

2.1.2　代码实现

```
CREATE FUNCTION GetStudentInfo()          -- 创建无参数函数，返回学号姓名+
RETURNS VARCHAR(50)
AS
BEGIN
    RETURN '马福泉+23336179'
END
GO
SELECT dbo.GetStudentInfo() AS StudentInfo;      -- 调用函数
```

2.1.3　运行结果

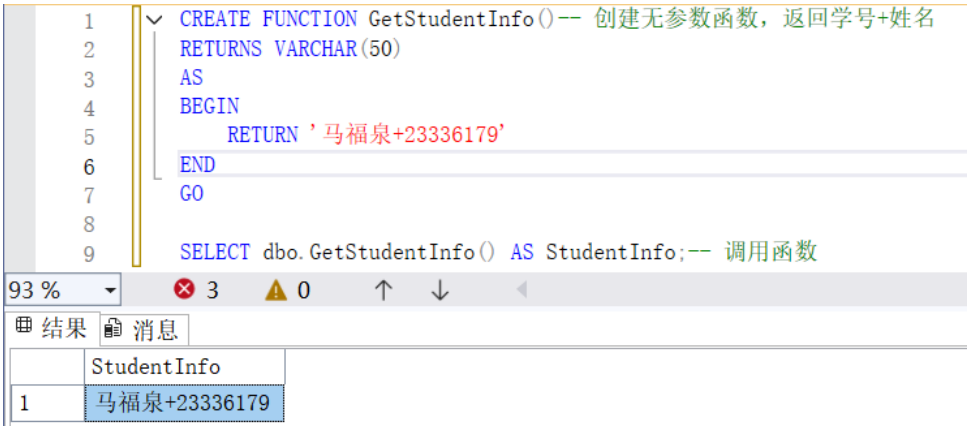


图 2: 无参数函数运行结果

2.2　有参数的存储过程或函数

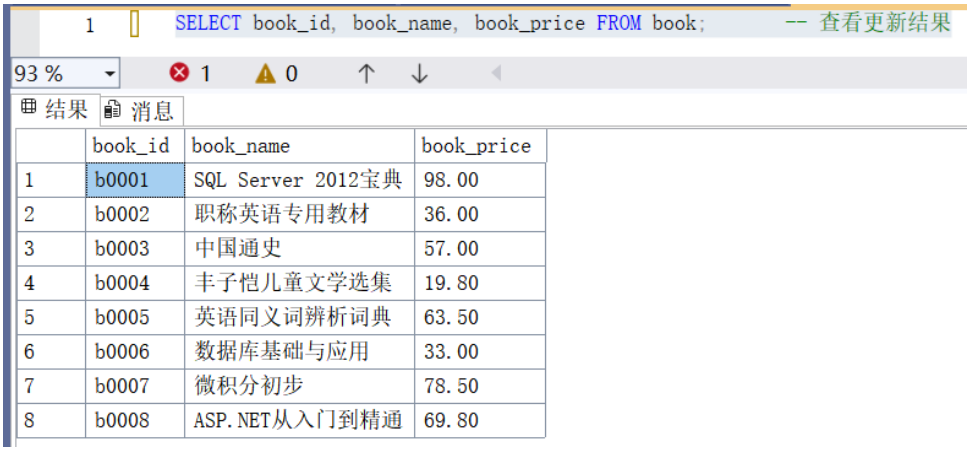
2.2.1　更新图书表价格

实验目标　将数据库中的图书表book_price更新为指定价格。

```
UPDATE book SET book_price = 98.00 WHERE book_id = 'b0001';          -- 更新图书表价
    格
UPDATE book SET book_price = 36.00 WHERE book_id = 'b0002';
UPDATE book SET book_price = 57.00 WHERE book_id = 'b0003';
UPDATE book SET book_price = 19.80 WHERE book_id = 'b0004';
UPDATE book SET book_price = 63.50 WHERE book_id = 'b0005';
UPDATE book SET book_price = 33.00 WHERE book_id = 'b0006';
UPDATE book SET book_price = 78.50 WHERE book_id = 'b0007';
```

```sql
UPDATE book SET book_price = 69.80 WHERE book_id = 'b0008';

SELECT book_id, book_name, book_price FROM book;        -- 查看更新结果
```



```
1  SELECT book_id, book_name, book_price FROM book;   -- 查看更新结果
```

93 % ⊗ 1 ⚠ 0 ↑ ↓

结果 消息

	book_id	book_name	book_price
1	b0001	SQL Server 2012宝典	98.00
2	b0002	职称英语专用教材	36.00
3	b0003	中国通史	57.00
4	b0004	丰子恺儿童文学选集	19.80
5	b0005	英语同义词辨析词典	63.50
6	b0006	数据库基础与应用	33.00
7	b0007	微积分初步	78.50
8	b0008	ASP.NET从入门到精通	69.80

图 3: 图书表价格更新结果

运行结果

2.2.2　查找学生所读图书

实验目标　定义一个存储过程或函数，找出某个同学所读的所有图书名。

```sql
CREATE PROCEDURE GetStudentBooks
    @StudentName VARCHAR(50)
AS
BEGIN
    SELECT b.book_name
    FROM book b
    INNER JOIN record r ON b.book_id = r.book_id
    INNER JOIN reader rd ON r.reader_id = rd.reader_id
    WHERE rd.reader_name = @StudentName;
END
GO

EXEC GetStudentBooks @StudentName = '谢嫣然';        -- 调用存储过程
```

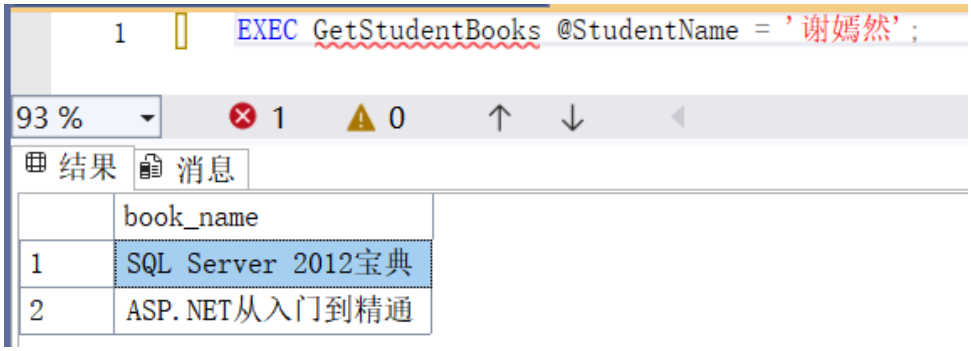


图 4: 查找学生所读图书结果

运行结果

2.2.3 图书价格调整

实验目标 定义一个存储过程，根据discount参数调整图书价格。

```
CREATE PROCEDURE AdjustBookPrice          -- 创建存储过程，调整图书价格
    @Discount DECIMAL(3,2)
AS
BEGIN
    CREATE TABLE #TempBookPrice (         -- 创建临时表存储原始价格，避免相互影响
        book_id CHAR(8),                  -- 修改为CHAR(8)匹配实际表结构
        original_price MONEY               -- 修改为类型匹配实际表结构MONEY
    );

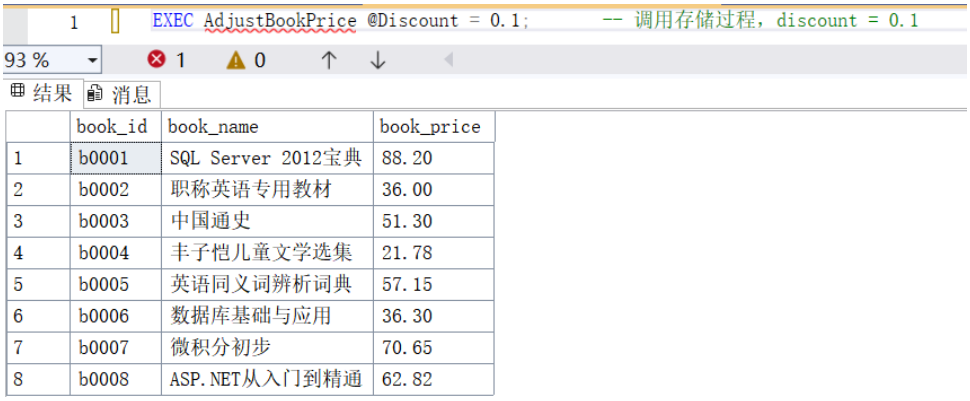
    INSERT INTO #TempBookPrice (book_id, original_price)          -- 插入原始价格
    SELECT book_id, book_price FROM book;

    UPDATE book           -- 更新价格小于等于的图书35
    SET book_price = t.original_price * (1 + @Discount)
    FROM book b
    INNER JOIN #TempBookPrice t ON b.book_id = t.book_id
    WHERE t.original_price <= 35;

    UPDATE book           -- 更新价格大于的图书55
    SET book_price = t.original_price * (1 - @Discount)
    FROM book b
    INNER JOIN #TempBookPrice t ON b.book_id = t.book_id
    WHERE t.original_price > 55;

    DROP TABLE #TempBookPrice;        -- 删除临时表
    END

EXEC AdjustBookPrice @Discount = 0.1;         -- 调用存储过程，discount = 0.1
```



	book_id	book_name	book_price
1	b0001	SQL Server 2012宝典	88.20
2	b0002	职称英语专用教材	36.00
3	b0003	中国通史	51.30
4	b0004	丰子恺儿童文学选集	21.78
5	b0005	英语同义词辨析词典	57.15
6	b0006	数据库基础与应用	36.30
7	b0007	微积分初步	70.65
8	b0008	ASP.NET从入门到精通	62.82

图 5: 图书价格调整结果

结果展示

book_id	book_name	调整后预期价格
b0001	SQL Server 2012宝典	98.00 × (1-0.1) = 88.20
b0002	职称英语专用教材	36.00
b0003	中国通史	57.00 × (1-0.1) = 51.30
b0004	丰子恺儿童文学选集	19.80 × (1+0.1) = 21.78
b0005	英语同义词辨析词典	63.50 × (1-0.1) = 57.15
b0006	数据库基础与应用	33.00 × (1+0.1) = 36.30
b0007	微积分初步	78.50 × (1-0.1) = 70.65
b0008	ASP.NET从入门到精通	69.80 × (1-0.1) = 62.82

和实际输出结果一致

2.2.4　查找师承关系

实验目标　根据导师关系表，输入姓名，查找该学生的师承关系链。

```
CREATE TABLE advisor (                              -- 创建导师关系表
    MentorName VARCHAR(50) NOT NULL,                -- 导师姓名
    StudentName VARCHAR(50) NOT NULL                -- 学生姓名
);

INSERT INTO advisor (MentorName, StudentName) VALUES
('Alice', 'Bob'),
('Alice', 'Carol'),
('David', 'Alice'),
('Mary', 'David'),
('Susan', 'Mary'),
('John', 'Susan');

CREATE FUNCTION GetAdvisorChain(@StudentName VARCHAR(50))        -- 创建递归查询函
    数，查找师承关系链
RETURNS VARCHAR(500)
AS
```

```sql
BEGIN
    DECLARE @Result VARCHAR(500) = '';

    WITH AdvisorChain AS (     -- 使用递归查找师承关系链CTE
        SELECT MentorName, 1 AS level
        FROM advisor
        WHERE StudentName = @StudentName

        UNION ALL

        SELECT a.MentorName, ac.level + 1
        FROM advisor a
        INNER JOIN AdvisorChain ac ON a.StudentName = ac.MentorName
    )
    SELECT @Result = COALESCE(     -- 将结果连接成字符串
        STUFF((
            SELECT ', ' + MentorName
            FROM AdvisorChain
            ORDER BY level
            FOR XML PATH(''), TYPE
        ).value('.', 'VARCHAR(MAX)'), 1, 2, ''),
        '无导师信息'
    );

    RETURN @Result;
END

-- 查询
SELECT dbo.GetAdvisorChain('Mary') AS AdvisorChain;       -- 结果应为Susan, John
```

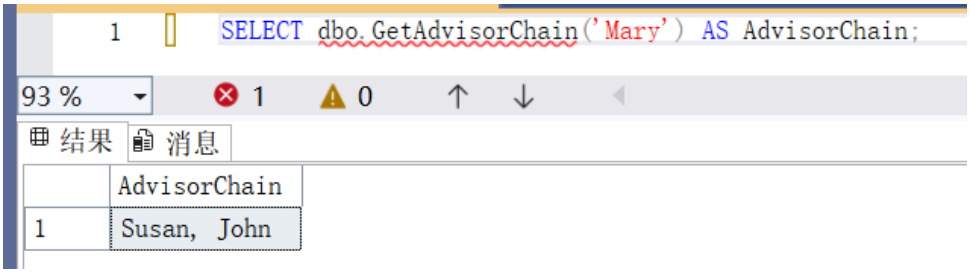


图 6: 师承关系查找结果

运行结果

2.3　游标实现图书总价计算

2.3.1　实验目标

定义一个存储过程或函数，用游标的方式计算所有图书的总价。

2.3.2 代码实现

```
-- 创建存储过程，使用游标计算图书总价
CREATE PROCEDURE CalculateTotalPriceWithCursor
AS
BEGIN
    DECLARE @TotalPrice MONEY = 0;   -- 改为类型匹配表结构MONEY
    DECLARE @CurrentPrice MONEY;     -- 改为类型匹配表结构MONEY
    DECLARE @BookName VARCHAR(100);
    DECLARE @BookId CHAR(8);         -- 添加图书，匹配表结构ID

    -- 声明游标，包含图书和名称ID
    DECLARE book_cursor CURSOR FOR
    SELECT book_id, book_name, book_price FROM book;

    OPEN book_cursor;        -- 打开游标

    -- 获取第一条记录
    FETCH NEXT FROM book_cursor INTO @BookId, @BookName, @CurrentPrice;

    WHILE @@FETCH_STATUS = 0         -- 循环处理每条记录
    BEGIN
        SET @TotalPrice = @TotalPrice + @CurrentPrice;
        PRINT '图书ID: ' + @BookId + ', 名称: ' + @BookName + ', 价
            格: ' + CAST(@CurrentPrice AS VARCHAR(20));

        -- 获取下一条记录
        FETCH NEXT FROM book_cursor INTO @BookId, @BookName, @CurrentPrice;
    END

    CLOSE book_cursor;       -- 关闭游标
    DEALLOCATE book_cursor; -- 释放游标

    PRINT '所有图书总价: ' + CAST(@TotalPrice AS VARCHAR(20));
    SELECT @TotalPrice AS TotalPrice;
END

EXEC CalculateTotalPriceWithCursor;      -- 调用存储过程
```

2.3.3 运行结果

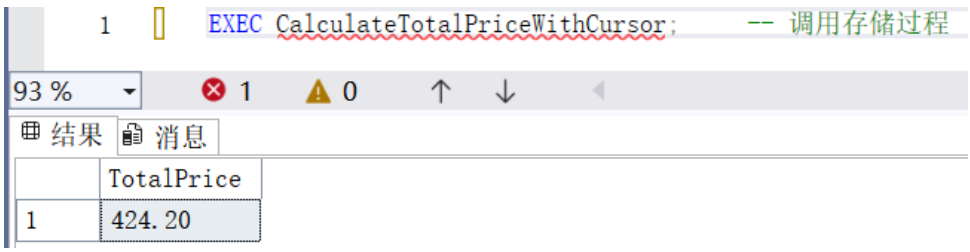


图 7: 使用游标计算图书总价结果

预期总价： $88.20 + 36.00 + 51.30 + 21.78 + 57.15 + 36.30 + 70.65 + 62.82 = 424.20$

预期结果与输出一致

2.4 实验总结

2.4.1 遇到的问题及解决方案

在进行图书价格调整实验时，遇到了价格变化太大被阻止的问题。这是由于之前的实验中设置了触发器来限制价格的大幅变动。

问题描述 执行价格调整存储过程时出现错误，系统阻止了价格的大幅变动。

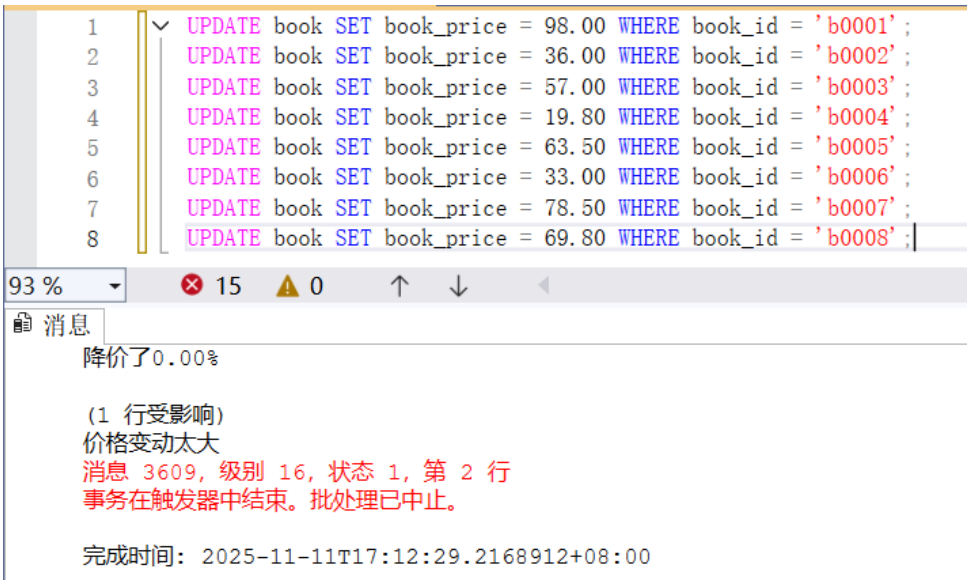


图 8: 触发器阻止价格变动错误截图

解决方案 需要先查找并删除相关的触发器，然后重新执行价格调整操作。

```
-- 方案一：查找并删除触发器
-- 1. 先查找图书表上的触发器名称
SELECT
    name AS TriggerName,                                    -- 触发器名称
    OBJECT_NAME(parent_id) AS TableName,                    -- 关联表名
    OBJECT_DEFINITION(object_id) AS TriggerDefinition    -- 触发器定义
FROM sys.triggers
WHERE parent_id = OBJECT_ID('book');

-- 2. 找到触发器名称后，删除触发器
DROP TRIGGER tr_book_price_after;                          -- 删除价格变动限制触发器

-- 方案二：临时禁用触发器（如果需要保留触发器）
DISABLE TRIGGER tr_book_price_after ON book;     -- 禁用触发器
```

问题2：SQL Server函数创建与调用批处理分隔问题　在SQL Server中，CREATE FUNCTION语句必须单独作为一个批处理，不能与其他语句（如SELECT）放在一起，否则会报语法错误。例如：

```
CREATE FUNCTION GetStudentInfo()
RETURNS VARCHAR(50)
AS
BEGIN
    RETURN '马福泉+23336179'
END
-- 调用函数
SELECT dbo.GetStudentInfo() AS StudentInfo;
```

上述写法会报错，因为CREATE FUNCTION和SELECT语句没有用GO分隔。

解决方案　应将CREATE FUNCTION语句与调用语句分开，使用GO分隔批处理或者将调用语句放在另一个查询中：

2.4.2　实验收获

通过本次实验，我掌握了：

1. 创建和调用无参数的存储过程和函数
2. 创建带参数的存储过程和函数，实现复杂的业务逻辑
3. 使用临时表避免数据更新时的相互影响
4. 实现递归查询功能
5. 使用游标遍历数据集进行计算

存储过程和函数是数据库编程的重要工具，能够封装复杂的业务逻辑，提高代码的复用性和维护性。